UnMock experiment - <PARTICIPANT_ID>

Unit tests often use mocks (for example, `@Mock`, `when(...)`, `verify(...)`) to simulate dependencies (i.e. other classes required by the class under test during the execution of the test). Each mock encodes assumptions about how the real dependency behaves, but the unit test never checks those assumption against the real implementations. UnMock is a framework that uses an LLM to transform existing unit tests into integration tests. It replaces mocked dependencies with proxies that call the real dependencies while checking the expectations expressed in the original tests.

In this experiment, you will use UnMock on provided unit tests from two open-source Java projects.

For each project, you will:

- Inspect the original unit test
- Generate an integration test using UnMock
- Review the generated code and make corrections where necessary
- Attempt to compile the generated integration test
- Attempt to run the generated integration test

Your main goal is to get the generated code to compile and run successfully in the target project. If it works without modifications, no changes are needed. Otherwise, please make a reasonable effort to resolve any issues you encounter. If you cannot resolve a problem after trying, make a note of it and move on.

After working on both projects, you will complete one questionnaire about your experience.

We are evaluating UnMock, not your programming skills. Your experience and feedback, including any issues you encounter, will help us understand how well the tool works and where it can be improved.

Experiment Setup

1. Check the prerequisites

Make sure you have installed:

- **Java JDK 21**
- **Apache Maven 3.9+**

Check installed versions:

```
java -version
mvn -version
```

2. Download and configure UnMock

Clone the UnMock repository:

```
git clone https://github.com/FedericoValeri/unmock4j.git
cd unmock4j
```

Create a `.env` file in the root of `unmock4j` with the following contents:

```
MODEL=gpt-6-luna
PROVIDER=openai
OPENAI_API_KEY=secret_key
```

Replace `secret_key` with the following openai api key:

- **OPENAI API KEY**: `<OPENAI_API_KEY>`

The OpenAI API key provided to you is intended exclusively for testing the Unmock project as part of this research activity. It is valid for one day and must not be used for any other purpose, shared with third parties, or used in other applications. Please keep the key confidential and use it only for the UnMock evaluation.

3. Download a target project and its input files

Download each target project into a directory of your choice:

- `<PROJECT_1_NAME>` (release `<PROJECT_1_VERSION>`): `<PROJECT_1_RELEASE_URL>`
  - SUT.txt: `<PROJECT_1_SUT_URL>`
  - UNIT.txt: `<PROJECT_1_UNIT_URL>`
  - DEPENDENCIES.txt: `<PROJECT_1_DEPENDENCIES_URL>`
- `<PROJECT_2_NAME>` (release `<PROJECT_2_VERSION>`): `<PROJECT_2_RELEASE_URL>`
  - SUT.txt: `<PROJECT_2_SUT_URL>`
  - UNIT.txt: `<PROJECT_2_UNIT_URL>`
  - DEPENDENCIES.txt: `<PROJECT_2_DEPENDENCIES_URL>`

Also download the related `SUT.txt`, `UNIT.txt`, `DEPENDENCIES.txt` files. These are the inputs UnMock needs to turn a unit test into an integration test:

- **`UNIT.txt`** contains the Mockito-based JUnit unit test to be transformed. It is the main input: UnMock examines the mocked dependencies, their configured behavior, and the interactions being verified.
- **`SUT.txt`** identifies the system under test, i.e the production class exercised by the unit test.
- **`DEPENDENCIES.txt`** contains source code for the mocked dependency types (interfaces or classes). UnMock uses their declarations, method signatures, and type information to construct proxies.

Experiment Procedure

Complete the following steps for each target project.

1. Inspect the original unit test

Open the unit test provided in `UNIT.txt` and try to understand it by identifying:

- The class under test (also given in `SUT.txt`).
- The dependencies that are mocked (for example, fields annotated with `@Mock`).
- The behavior configured for the mocks (for example, `when(...).thenReturn(...)`).
- The interactions checked by the test (for example, `verify(...)`).

2. Provide the input files to UnMock

Copy the three downloaded files into:

```
unmock4j/src/main/resources/prompts/human/
```

The directory has to contain the files with exactly these names:

```
SUT.txt
UNIT.txt
DEPENDENCIES.txt
```

Overwrite existing files with the same names. All these files have to correspond to the project you are currently evaluating.

3. Check the interaction with the LLM

From the root of `unmock4j`, run the following commands to check the connection to the LLM, in order to verify UnMock configuration has been done properly.

```
mvn clean package
java -jar target/unmock-1.0-SNAPSHOT.jar --llm-check
```

If the check fails, please check your internet connection and the `.env` file. If the problem persists, contact `<CONTACT_EMAIL>`.

4. Generate the integration test

From the root of `unmock4j`, run the application:

```
java -jar target/unmock-1.0-SNAPSHOT.jar
```

UnMock generates an integration test class and one or more proxy classes (with names ending in `_Proxy`) in its `gen/` directory.

5. Compile the integration test

From the root of `unmock4j`, run the appropriate checking script for your operating system. Replace `<local_repo_path>` with the path of the target project repository in your machine.

- **Windows** (PowerShell):

```
.\scripts\check-gen.ps1 -Repo <local_repo_path>
```

- **Linux / macOS** (bash):

```
bash scripts/check-gen.sh --repo <local_repo_path>
```

This script will move the UnMock generated files into the target open-source project and compile them. The output of the script includes:

- `Module`: the target project's Maven module.
- `Target`: the folder where the generated files were copied.
- `copy ... -> ...`: the destination of each copied file.
- The Maven compilation result and any compilation errors.

Important: This script checks whether the generated code compiles; it **does not run the integration test**.

6. Inspect and, if necessary, fix the generated code

Your main goal is to inspect the integration test and proxy classes generated by UnMock and make any necessary changes to get them to **compile successfully** in the target project.

Follow these steps:

1. Open the cloned target project in an IDE, such as IntelliJ IDEA or Eclipse.
2. Find the generated integration test and proxy classes using the destination paths (`->`) shown by the script. Inspect them even if compilation succeeded.
3. **If the code does not compile**, fix the problems you can identify. For example:
    - Incorrect imports, method signatures, or constructor calls.
    - Missing or incorrect dependency initialization.
    - Configuration points marked by `REAL_CONFIGURED_INSTANCE`, where a real dependency instance must be provided.
4. Make **only the changes needed** to get the generated test compiling, and preserve the intentions of the original unit test as far as possible.
5. Recompile the code after making changes. From the root of the target project, run the following command, replacing `<module>` with the module shown in the checking script's output: `mvn clean test-compile -pl <module> -am`

Note: Do not run `check-gen` again after editing the copied files: it would overwrite your changes.

7. Run the integration test

Once step 6 is completed successfully, execute the generated integration test. If you cannot fix any issue, record the error and what you tried. You can still complete the experiment and questionnaire.

Questionnaire

Complete the questionnaire once, after finishing both projects. It takes approximately 5 minutes.

- Enter your participant code, which you find below. Your name is not collected.
  - **Code**: `<PARTICIPANT_CODE>`
- Rate each statement from 1 (strongly disagree) to 4 (strongly agree), based on the tasks you just completed.
- The open-ended questions are optional but very helpful. In particular, describe what you changed in the generated code and why.

Link to questionnaire: `<QUESTIONNAIRE_URL>`